ARVR PROJECT | TECHNICAL REFERENCE DOCUMENT

SLAM-Based AR Medicine Identifier with Voice-Guided Accessibility Mode

Deep project context, improved pipeline, functional design and a feasibility-checked technology stack

What this document is

A deeper, design-level companion to the short project overview. It explains **what the system really has to do**, stress-tests the original design, proposes an **improved end-to-end pipeline**, describes **exactly how the app works from the user's point of view**, and recommends a **technology stack that is realistic for a student team** — including what to cut or stage so the project actually finishes.

How to read it

Sections 1–3 give the understanding and the critique. Sections 4–9 are the technical core (architecture, pipelines, accessibility, functional behaviour, data). Sections 10–14 are the build plan (stack, roadmap, evaluation, safety, risks). Section 15 supports your literature survey. Appendices contain schemas, tested snippets and UI specifications.

Version 1.0 | October 2026 | Companion to the project overview PDF

Contents

1. Executive Summary 4

2. Problem Deep-Dive, Users and Requirements 5

2.1 What actually goes wrong with medicine labels . . . 5

2.2 Target users (design personas) . . . 5

2.3 Functional requirements . . . 5

2.4 Non-functional requirements (with proposed targets) . . . 6

3. Critical Review of the Original Design 7

4. Reference Architecture (v2) 8

4.1 Three loops, one shared scene state . . . 8

4.2 On-device first, cloud optional . . . 9

5. Improved Recognition Pipeline ("what is this?") 10

5.1 Stage-by-stage specification . . . 10

5.2 Original vs improved pipeline at a glance . . . 12

6. AR Pipeline in Depth ("how it is shown") 13

6.1 What SLAM provides — and what it does not . . . 13

6.2 Lifting a 2D detection into 3D . . . 13

6.3 Anchoring modes: world-locked vs object-following . . . 14

6.4 Pose-at-capture-time compensation . . . 14

6.5 Panel design and layout . . . 14

6.6 Tracking-failure handling . . . 15

6.7 Performance and thermal management . . . 15

6.8 Curved bottle labels . . . 15

7. Accessibility Layer in Depth 16

7.1 Scan-guidance engine (the missing piece in the overview) . . . 16

7.2 Point-and-hold automatic trigger . . . 16

7.3 Speech design . . . 16

7.4 Synchronised large-print captions . . . 16

7.5 Screen-reader compatibility (important and often missed) . . . 16

7.6 Design guidance for elderly users . . . 17

7.7 Voice commands . . . 17

8. How the System Works, Functionally 18

8.1 End-to-end user journey . . . 18

8.2 Worked example (illustrative data) . . . 18

8.3 Timing budget (illustrative targets, to be measured) . . . 19

8.4 Edge cases and alternative flows . . . 19

8.5 Scan history and expiry reminders . . . 19

9. Data and Knowledge Layer 21

9.1 Knowledge-base strategy: closed world first . . . 21
9.2 Candidate data sources (check each licence) . . . 21
9.3 Two-hop model: brand → composition → monograph . . . 21
9.4 Matching in practice . . . 21
9.5 Content governance . . . 22
9.6 Expiry parsing rules . . . 22
10. Technology Stack — Brainstormed and Feasibility-Checked 23
10.1 Three candidate platform stacks . . . 23
10.2 Recommended stack by layer (Stack A, with iOS equivalents) . . . 24
10.3 ML, data and tooling . . . 25
10.4 Device requirements and test matrix . . . 25
10.5 Considered and not recommended (as the primary path) . . . 25
11. Implementation Plan 26
11.1 Module structure (Stack A) . . . 26
11.2 Phased roadmap (assuming a ~12-week window — rescale to your calendar) . . . 26
11.3 MVP cut line . . . 26
12. Evaluation Plan 27
12.1 Metrics . . . 27
12.2 Experiments / ablations . . . 27
12.3 User study design . . . 27
13. Safety, Privacy, Ethics and Regulatory Notes 28
14. Risk Register 28
15. Literature Survey Guide 29
15.1 Comparison table to build in your survey . . . 29
15.2 Gap analysis and novelty statement (to refine after your search) . . . 30
15.3 Suggested limitations section . . . 30
16. Future Work 31
Appendix A — Data Schemas 32
Appendix B — Key Algorithms (tested snippets) 32
Appendix C — UI and Accessibility Specification (starting values) 33
Appendix D — Glossary 34

1. Executive Summary

The idea in one sentence. A phone camera continuously recognises a medicine strip or bottle, works out what it is from the printed text, and then *speaks* and *shows* the key facts (what it is for, how the label says to take it, main warnings, expiry status) attached to the physical object — so a person who struggles to read small print never has to.

Why it is a good ARVR project. It joins three research areas that are usually treated separately: (1) markerless mobile AR / SLAM, (2) scene-text recognition on difficult packaging, and (3) accessibility-first multimodal interaction. The engineering challenge is real (foil glare, hand-held jitter, close-range tracking) and the social value is easy to explain.

What this document changes compared with the overview

#	Original overview	Recommendation in this document	Why
1	World-anchored panels persist even when the object leaves the frame	Two anchoring modes: **world-locked** when the object is stationary, **object-following** when it is hand-held	A hand-held strip moves relative to the world, so a fixed world anchor goes stale
2	ARCore Augmented Images / ARKit image anchors to lock onto the medicine	**Open-set, text-driven localisation**: OCR boxes (+ optional detector) → ROI → 3D position by raycast or size-prior depth	Image targets must be pre-registered per product; thousands of SKUs and glossy foil do not fit that model
3	Single-pass OCR → regex → fuzzy match	**Multi-frame fusion**, glare/blur gating, barcode channel, cross-field verification, calibrated confidence gate	Foil strips are the hardest OCR case; silent mis-identification is a safety problem
4	Visual AR overlay is the main output	**Eyes-free first**: scan-guidance audio/haptics, auto-trigger, TTS-led output, overlays as a parallel channel	A user who cannot see the screen cannot aim at the object using the screen
5	Product list: dosage, side effects, usage, expiry	Same facts, but framed as **"label information"**, ordered **identity → expiry verdict → how to use → warnings**	Avoids implying a personalised dose; puts the most actionable fact first
6	Mostly cloud/licensed DB suggested	**Offline curated closed-world DB** (a few hundred common medicines) + optional online fallback	Feasible to verify; works without network; avoids licensing traps
7	Unity/AR Foundation implied as default	**Native Android (Kotlin + ARCore + ML Kit)** as primary stack; Unity and iOS as alternatives (Section 10)	Shortest path to a working, accessible, high-quality OCR build

DESIGN PRINCIPLE

The single most important design principle: in a medication context a confident wrong answer is far worse than an honest "I am not sure — please hold the strip again." Every part of the improved pipeline (quality gating, multi-frame voting, verification against a trusted database, confirmation prompts) exists to make the system *fail safe* rather than *fail silently*.

Recommended build, in one paragraph

Native Android app (Kotlin, Jetpack Compose) using ARCore for SLAM tracking, ML Kit for on-device OCR and barcode reading, OpenCV for image-quality metrics and geometry, an offline SQLite drug knowledge base, Android TextToSpeech with word-level callbacks for caption sync, and WorkManager for expiry reminders. Perception runs asynchronously at a few Hz while rendering runs at 30–60 Hz; results are lifted into 3D using the camera pose *at the time the frame was captured*. iOS (ARKit + Vision) is a clean second port. Anything beyond the MVP (custom detector, learned field extractor, cloud vision fallback, multilingual output) is staged as optional upgrades.

2. Problem Deep-Dive, Users and Requirements

2.1 What actually goes wrong with medicine labels

Medication errors are a recognised global patient-safety problem (the WHO's *Medication Without Harm* challenge, launched in 2017, targets severe, avoidable medication-related harm). Packaging contributes in specific, concrete ways that your system can address:

Failure mode	Typical situation	What the system must do
Misreading name or strength	Small print, low contrast, tablets removed from the carton and only a foil strip left	Read the strip text and state name + strength aloud and in large print
Look-alike / sound-alike names	Two drugs with similar names (for example metformin vs metronidazole)	Never match on name alone; verify strength and composition; use confirmation when two candidates are close
Expired or near-expiry medicine	Expiry printed as "EXP 08/27" at the edge of a strip	Parse expiry, compute status from today's date, announce it first when it matters
Forgotten instructions	Elderly user, many medicines, no leaflet	Plain-language "what it is for / how the label says to take it / key warnings"
Inaccessible interface	Visually impaired or low-dexterity user must aim, tap and read a screen	Eyes-free scanning, automatic trigger, spoken output, no precise taps

2.2 Target users (design personas)

Persona	Needs	Design implications
Elderly person on several prescriptions	Quick reassurance of what a strip is and whether it is in date; low tech confidence	One-step flow, large type, slow clear speech, no timeouts, forgiving of shaky hands
Visually impaired user (low vision or blind)	Eyes-free operation, aiming help, screen-reader compatibility	Audio/haptic guidance, automatic capture, every overlay mirrored as accessibility text
Caregiver or family member	Check medicines quickly, keep a history, get expiry alerts	Scan history, reminders, (future) sharing
General user (secondary)	Verify an unfamiliar strip	Same pipeline; optional detail view

2.3 Functional requirements

ID	Requirement	Priority
FR1	Detect a medicine strip, blister pack, carton or bottle label in the live camera view	Must
FR2	Read printed text (name, strength, composition, batch, expiry) on-device, in real time, with multi-frame refinement	Must
FR3	Match the reading to a verified drug knowledge base with a calibrated confidence score	Must
FR4	Present dosage-on-label, usage, key side effects and expiry status as AR overlays attached to the object	Must
FR5	Accessibility mode: spoken output + synchronised large-print captions, point-and-hold auto-trigger, haptic confirmation	Must
FR6	Eyes-free scan guidance (distance, glare, steadiness, tracking problems) through speech and haptics	Must
FR7	Store scan history locally; schedule expiry reminders (for example 30 days, 7 days, on the day)	Must
FR8	Ask for confirmation when confidence is medium or two candidates are close; never present low-confidence results as fact	Must
FR9	Barcode / QR / DataMatrix reading as an additional identification channel when present	Should
FR10	Voice commands: repeat, more detail, save, scan another, stop	Should
FR11	Graceful 2D fallback on devices without ARCore/ARKit support (same recognition, simpler overlay)	Should
FR12	Multilingual spoken output (English plus selected Indian languages)	Could

2.4 Non-functional requirements (with proposed targets)

The numeric targets below are **proposed starting hypotheses** to be confirmed in your evaluation, not established benchmarks.

Quality	Proposed target	Notes
Time to first result	Median ≤ 3 s from stable framing	Dominated by multi-frame fusion; measure on mid-range phones
Identification accuracy (closed-world DB, decent conditions)	Top-1 ≥ 90%	Report separately for strips, cartons, bottles
False-confident rate (auto-announced but wrong)	≤ 1%	**The headline safety metric** — optimise this before top-1
Overlay stability when object is still	≤ 3 px RMS jitter	Measured on screen with a tripod
Frame rate	≥ 30 FPS rendering while perception runs	Perception must never block rendering
Privacy	No image leaves the device by default	Cloud fallback only with explicit opt-in
Accessibility	WCAG-style contrast ≥ 7:1 for captions; touch targets ≥ 48 dp	Screen-reader compatible UI
Offline operation	Full core flow with no network	On-device DB, OCR and TTS voices

3. Critical Review of the Original Design

The overview is a sound starting point and its instinct — SLAM for "where", AI pipeline for "what" — is correct. The points below are places where the design would hit trouble in practice, with the resolution adopted in this document. Several are worth stating in your report as **explicit design insights**.

#	Observation	Why it matters	Resolution
F1	"Anchors persist even if the object leaves the frame."	True only if the object itself is stationary. A hand-held strip moves relative to the world, so a world-fixed anchor becomes wrong the moment the hand moves.	Two modes: **world-locked** (object still ≥ ~1 s) and **object-following** (hand-held, panels re-posed from tracked object pose). Never show stale overlays as if valid.
F2	Use ARCore Augmented Images / ARKit image anchors to lock onto the strip.	Image targets need a pre-registered reference image per product; ARCore databases hold on the order of a thousand images (check current docs) and prefer flat, feature-rich art. Foil strips are shiny, curved and visually near-identical across products.	Use **open-set, text-driven localisation**: OCR boxes and an optional lightweight detector give an ROI; lift it to 3D (Section 6). Keep image targets only as an optional demo-set booster.
F3	Single-frame OCR.	Specular glare on foil, embossed or low-contrast print, motion blur and tiny fonts make single-frame OCR unreliable.	Quality gate + **multi-frame fusion** + token voting; strips repeat the same text per cell, which gives free extra samples.
F4	Fuzzy match to the database decides the identity.	Fuzzy string matching can pick a plausible but wrong look-alike drug. This is the most dangerous failure.	Match on **brand AND composition**, verify strength, require a score margin over the runner-up, and ask for confirmation otherwise.
F5	"Dosage" as an overlay item.	The label shows a strength (500 mg); the correct *dose* is prescribed per person. Showing a "dosage" implies personalised advice.	Present **label information** ("strength", "label directions") and always defer to the prescriber/pharmacist.
F6	Camera assumptions.	AR sessions often default to fixed focus and a modest CPU-image resolution; strips are read at 15–30 cm; depth sensing is weak at close range.	Enable continuous autofocus, choose the highest usable CPU-image config, capture a high-res still for OCR when possible, and coach the user to a good distance.
F7	The AR overlay is the primary output.	A blind or very-low-vision user cannot use the screen to aim; "point-and-hold" assumes they can already frame the object.	**Eyes-free scan guidance** (speech, haptics, earcons) and auto-trigger; overlays are a parallel channel and mirrored to the screen-reader tree.
F8	Real-time pipeline implied to be synchronous.	OCR takes tens to hundreds of milliseconds; the camera moves meanwhile, so a result lifted with the *current* pose lands in the wrong place.	Asynchronous loops and **pose-at-capture-time compensation** (store camera pose per frame, use it when the result returns).
F9	Drug data from commercial or unclear sources.	No single free source offers Indian brand names plus patient-friendly clinical text; scraping commercial sites breaks their terms.	Two-hop knowledge base: brand → composition → monograph, curated for a closed set (for example a few hundred common medicines) with source and reviewer fields.
F10	SLAM needs texture; strips are small and reflective.	Close to a plain foil or palm, tracking quality can drop (reasons are exposed by ARCore as insufficient features, excessive motion or insufficient light).	SLAM tracks the *room*, not the strip — keep background in view; map tracking-failure reasons to spoken tips; degrade gracefully.

KEY INSIGHT

Is SLAM actually needed? Yes, but for a narrower reason than the overview implies. SLAM gives a stable 6-DoF camera pose at high rate, which lets you (a) place content in a consistent world frame, (b) correct for the latency of OCR by re-projecting results, (c) hold panels steady while the camera moves, and (d) anchor static objects robustly. It does *not* recognise the medicine and it does not track a hand-held object by itself. Presenting this honestly in your report ("SLAM + recognition + object pose, each doing what it is good at") is stronger than claiming SLAM solves tracking of the strip.

4. Reference Architecture (v2)

The system is organised in five layers. Each layer can be built, tested and replaced independently, which also makes the evaluation cleaner (for example, swapping the OCR engine without touching the AR code).

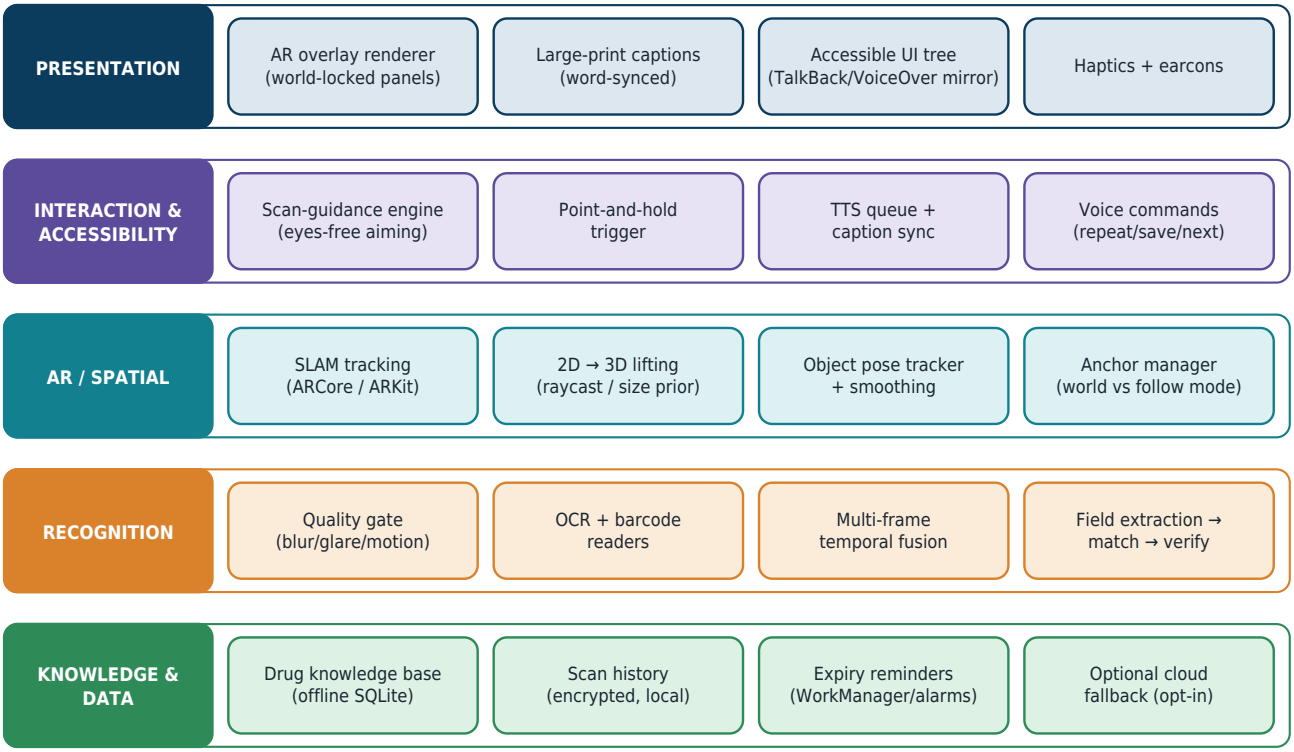


Figure 1. Layered architecture. Perception and AR run on separate loops; accessibility cuts across both.

4.1 Three loops, one shared scene state

A common mistake is to run OCR inside the AR render callback, which freezes the frame rate. Instead the app runs three loops that communicate through a thread-safe **scene state**:

- **Render loop (30–60 Hz):** reads the camera pose and the latest tracked object pose, updates panel transforms with smoothing, draws overlays and captions. It never waits for perception.
- **Perception loop (≈3–10 Hz depending on device):** takes the *latest* frame only (old frames are dropped), runs quality gating, OCR/barcode, fusion, extraction and matching. Results carry the **frame timestamp**.
- **Accessibility loop (event-driven):** plans guidance cues, manages a prioritised speech queue, synchronises captions and triggers haptics.

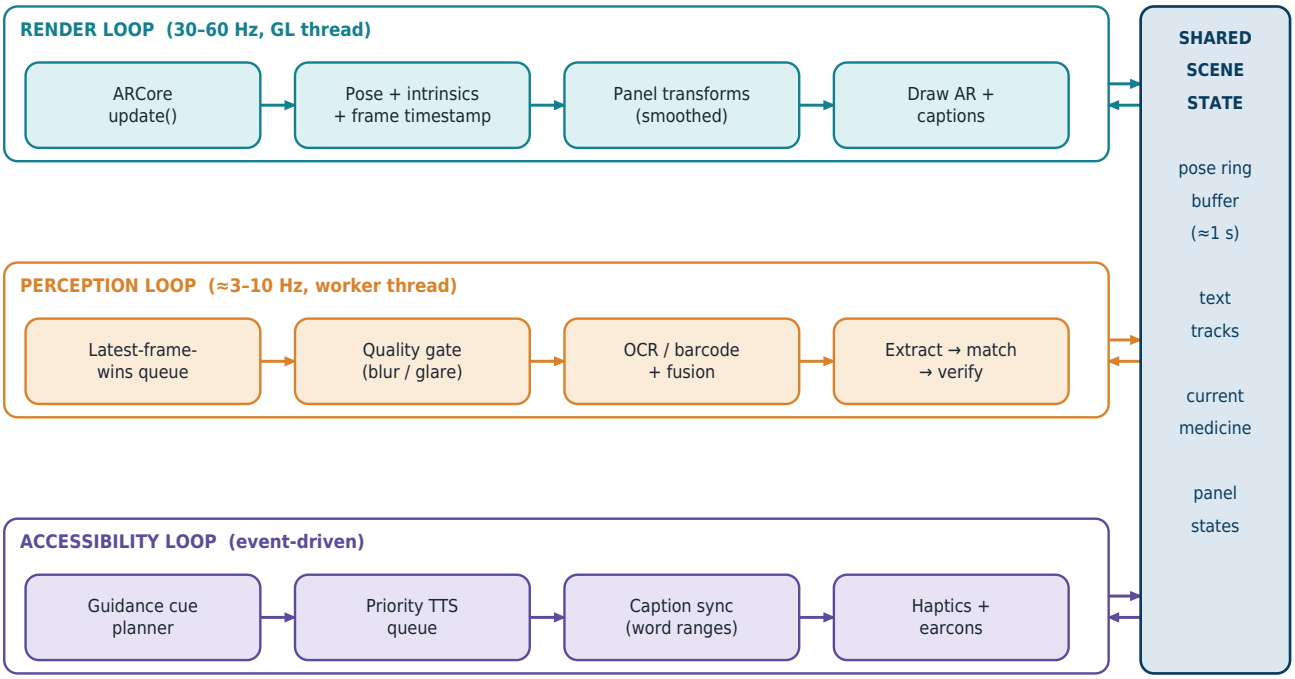


Figure 2. Concurrency model. A pose ring buffer (about one second) lets late OCR results be placed using the pose from their capture time.

4.2 On-device first, cloud optional

Aspect	On-device (default)	Optional cloud fallback
Privacy	Images and medicine history never leave the phone	Only a cropped label image, only with explicit consent each time
Latency	Tens of ms per OCR call; works in a pharmacy basement	Network round trip; only used when on-device confidence is low
Accuracy	Good on clear print; can struggle on foil glare	A vision-language model can read hard text, but its output is treated as *evidence* and verified against the DB, never as truth
Cost	Free at runtime	Per-request cost; rate-limit and cache

SAFETY

Rule for any AI-generated text: a language or vision-language model may help *read* a hard label, but every sentence spoken or shown about the medicine must come from the verified knowledge base. No generative model should author medical content shown to the user.

5. Improved Recognition Pipeline ("what is this?")

The overview pipeline (preprocess → OCR → regex → fuzzy match) is the right skeleton. The improved version adds the stages that matter most for foil packaging and for safety: quality gating, optional localisation, a barcode channel, temporal fusion, cross-field verification and a confidence gate with human confirmation.

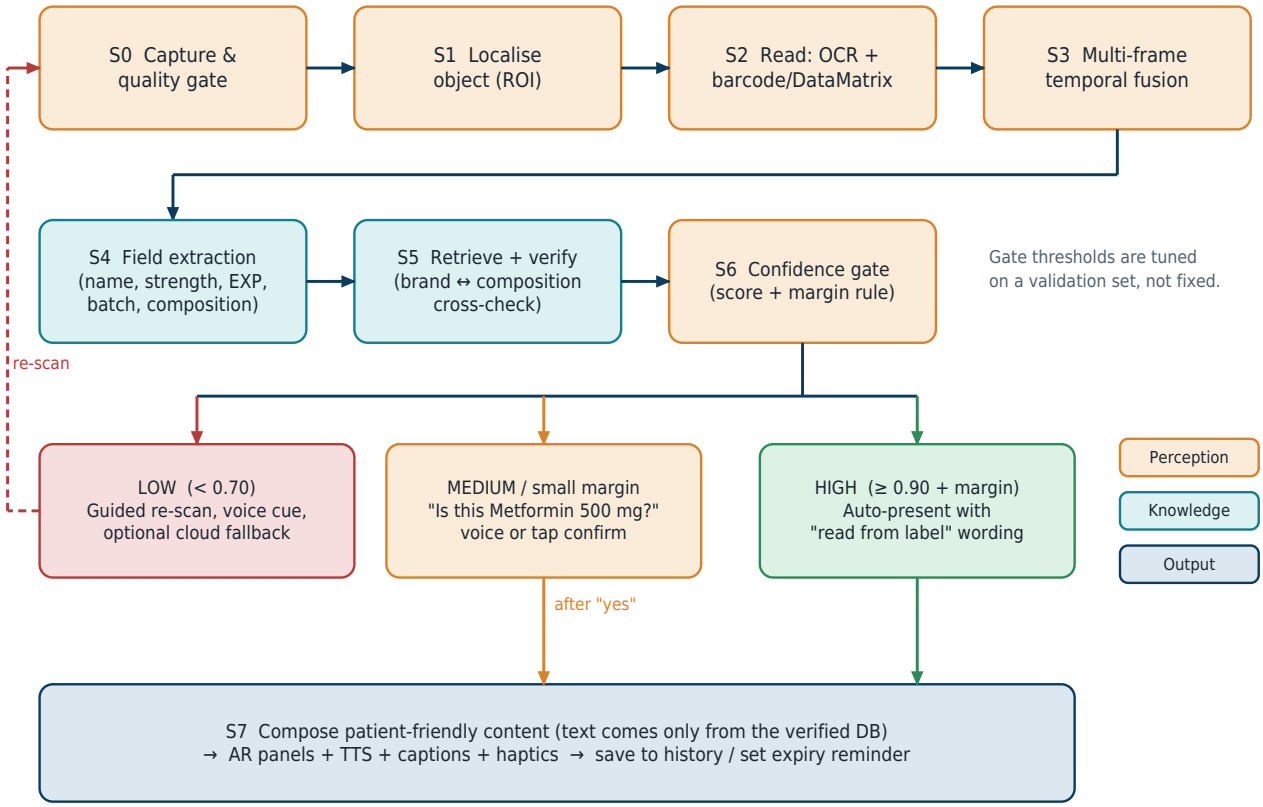


Figure 3. Improved recognition pipeline. The red dashed path returns the user to scanning with a specific spoken tip.

5.1 Stage-by-stage specification

S0 — Capture and quality gate

- **Inputs:** camera frames, gyroscope, AR tracking state.
- **Metrics computed cheaply per frame:** sharpness (variance of the Laplacian on the ROI), exposure (mean luminance and clipped-pixel fraction), glare (fraction of near-saturated pixels with low local texture inside the ROI), motion (gyro magnitude).
- **Behaviour:** only frames that pass thresholds are sent to OCR. A failed metric produces a specific cue ("Tilt the strip slightly — there is glare", "Move a little farther back", "Hold steady").
- **Camera setup:** continuous autofocus on; highest practical image size; when the platform allows it, capture a **high-resolution still** of the ROI once the frame is stable (ARKit offers high-resolution frame capture; on Android use the largest CPU image config or the shared-camera path).

S1 — Localise the object (ROI)

- **MVP:** skip a dedicated detector. Run text detection on the full frame and cluster the text boxes into an object ROI (the minimum-area rectangle around dense text).
- **Upgrade:** a small object detector (classes: strip, carton, bottle/label) gives a clean ROI, an object-class prior for physical size, and better crops for high-res capture. It needs a labelled dataset (Section 12).
- **Output:** ROI quadrilateral in image coordinates plus an object-class guess.

S2 — Read: OCR and barcode channels

- **OCR:** on-device scene-text recogniser returning lines with bounding boxes, confidences and (where available) per-element data. Latin script covers brand and generic names for most Indian packaging.
- **Barcode / QR / DataMatrix:** medicine cartons often carry a GS1-style code; a successful decode gives an identifier that can be looked up directly (very high precision). Strips frequently do not carry one, so this is a bonus channel, not a replacement. Regulatory requirements for codes on packaging have been phased in on some Indian medicines — verify the current rules for your target market before relying on them.
- **Both channels feed S3**; a barcode hit that resolves to a DB product can short-circuit to S5 verification.

S3 — Multi-frame temporal fusion

This is the biggest accuracy lever for foil. Text lines are **tracked** across frames by associating boxes after compensating for camera motion (using the AR pose or optical flow). Each track keeps a small bag of string hypotheses with confidences, and the system votes:

- A line is considered **stable** when the same string (edit distance ≤ 1) appears in at least 3 of the last 5 usable frames.
- Per-character voting after aligning near-identical strings recovers digits and letters that flicker (for example 0/O, 1/l, 5/S, rn/m).
- On blister strips the same text repeats in every cell; treat each repeat as an independent sample.
- Exit condition: all *required* fields (name or composition, strength) stable, or a timeout that triggers guidance.

S4 — Field extraction

Convert stable text into structured fields using rules first, with a learned extractor as an optional upgrade.

Field	Method	Notes
Brand / product name	Largest-height text lines near the top; exclude lines dominated by digits or boilerplate	Box height is a good proxy for font size
Strength	Regex: number + unit (mg, mcg, g, ml, IU, %) and combinations such as "500 mg + 125 mg"	Normalise units; keep original string
Composition / generic	Lines after "Each … tablet contains:" or ingredient + "IP/BP/USP" patterns	Primary key for brand-independent matching
Expiry	Keyword-anchored regex (EXP, EXPIRY, USE BEFORE) with month/year formats; digit-confusion repair; end-of-month rule	Tested snippet in Appendix B
Batch / MFG / MRP	Keyword-anchored (B.No., Batch, MFG, MRP)	MFG is not expiry — never confuse them
Manufacturer	"Manufactured/Marketed by" lines	Used as a weak confirming signal
Upgrade: learned tagger	Token classifier (NER-style) trained on labelled OCR output, optionally using box positions	Compare against rules in your ablation study

S5 — Retrieve and verify

1. **Normalise** text (case, punctuation, common OCR confusions).
2. **Retrieve candidates** from two keys: brand-name key and composition key (exact match first, then character n-gram / fuzzy retrieval of the top ~20–50).
3. **Re-rank** with similarity features: Jaro-Winkler / Levenshtein ratio, phonetic code (for example Double Metaphone), strength equality, composition overlap, manufacturer match.
4. **Cross-field verification:** the brand's DB composition must agree with the composition read from the label. Agreement is strong evidence; disagreement forces confirmation.
5. **Barcode override:** an exact identifier match to a DB product sets the score to the maximum, subject to the same agreement check.

S6 — Confidence gate (fail-safe decision)

Condition	Action
Score ≥ 0.90 **and** margin over runner-up ≥ 0.10 **and** strength verified	**Auto-present**, phrased as "Reads as ..." (it is a reading of the label, not a certificate)
0.70 ≤ score < 0.90 **or** small margin **or** strength unverified	**Confirm:** "I think this is Metformin 500 milligram. Is that right?" (voice or large tap)
Score < 0.70 or no stable fields	**Guided re-scan** with a specific cue; after repeated failure offer manual/voice entry or the opt-in cloud fallback

The thresholds are **starting heuristics**: fit them on a validation set so that the false-confident rate meets your target (Section 12). A simple weighted score to start from is given in Appendix B.

S7 — Compose content

Content is assembled from templates and DB fields only, in plain language, in a fixed order: **identity → expiry verdict → how the label says to take it → key warnings → common side effects**. A short "summary" (about 10 seconds of speech) comes first; details are given on request ("tell me more"). Look-alike drug names can be displayed with **tall-man lettering** (for example metFORMIN / metroNIDAZOLE) to help visual discrimination.

5.2 Original vs improved pipeline at a glance

Aspect	Original	Improved (v2)
Frames used	One frame	Many frames, voted
Glare / blur	Generic "correction"	Measured, gated, and turned into a spoken tip
Identification keys	Name fuzzy match	Brand + composition + strength + barcode, cross-verified
Wrong-answer handling	Not specified	Margin rule, confirmation step, honest "reads as" wording
Object localisation	Image target / plane detection	Text-cluster ROI (+ optional detector), 3D lift by raycast or size prior
Output ordering	Name → dosage → expiry → side effects	Identity → expiry verdict → how to use → warnings; summary then detail

6. AR Pipeline in Depth ("how it is shown")

6.1 What SLAM provides — and what it does not

SLAM / visual-inertial tracking provides	It does not provide
6-DoF camera pose at high rate (camera images fused with the IMU)	Any knowledge of what the strip is (that comes from OCR/vision)
Camera intrinsics, tracking state and failure reasons	The pose of a *hand-held* object — the object moves independently of the world
Sparse feature points and detected planes (and optionally depth) to raycast against	Reliable depth for small objects at 15–30 cm; depth is weakest at very close range
Drift-corrected anchors for *static* content	Anchors you can re-position: platform anchors are fixed in the world and cannot be moved

6.2 Lifting a 2D detection into 3D

The perception loop returns an ROI in image pixels. To attach panels you need a 3D position. Three methods, in order of preference, with the pitfall that makes the third one essential for hand-held use:

Method	How	Works when	Fails when
M1 Depth raycast	Cast a ray through the ROI centre against the depth map (depth-from-motion, ToF or LiDAR)	Object ≥ ~0.3–0.5 m away; device supports depth	Very close, small, shiny objects
M2 Plane / feature raycast	Raycast against detected planes or feature points	Strip lies flat on a table (the ray hits the strip's surface)	**Hand-held strip: the ray passes the strip and hits the table behind it**
M3 Size-prior depth + pose	$Z = f \cdot W_{real} / W_{px}$ using camera intrinsics and an expected physical width for the object class; optionally solve a planar pose (PnP) from the ROI corners	Always available; error tolerance is fine for panel placement (±15–20%)	Wrong class prior (carton mistaken for strip) — mitigated by the detector or by user-confirmed class

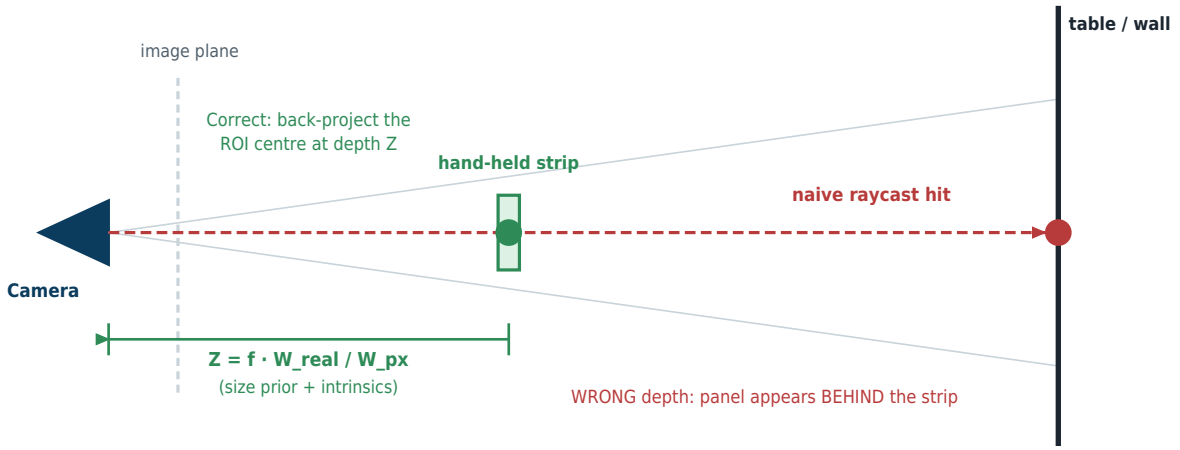


Figure 4. Why naive raycasting misplaces panels for hand-held objects, and how size-prior depth fixes it (side view).

DESIGN DECISION
Practical recipe: try M2 only when the object has been stationary and the hit lies within a small depth window of the M3 estimate; otherwise trust M3. Measure 5–10 real strips and cartons once to set the width priors (store ranges, not single numbers). Use the camera intrinsics reported by the AR session.

6.3 Anchoring modes: world-locked vs object-following

	WORLD-LOCKED mode	OBJECT-FOLLOWING mode
Trigger	Object pose velocity ≈ 0 for ≥ ~1 s (strip on the table or held very still)	Object moving (hand-held); default on start
Implementation	Create a real platform **Anchor** at the filtered object pose; the SLAM back-end keeps correcting it against drift	Own transform in world coordinates, re-posed from each new measurement and smoothed (platform anchors cannot be moved)
Behaviour when object leaves view	Panels remain in place — this is the genuine "persistence" benefit	Panels fade to a ghost style and dock to a screen-space tray after ~1 s; never pretend to be attached
Smoothing	None needed (anchor is stable)	One-Euro or Kalman filter on position, spherical interpolation on orientation; a deadband to suppress micro-jitter

Between OCR updates (every 100–300 ms) the object pose is **propagated**: with optical flow on the ROI or simply by assuming constant pose relative to the camera for a short interval, and corrected when a new measurement arrives. This keeps panels visually continuous even though recognition is slow.

6.4 Pose-at-capture-time compensation

When a frame is sent to the perception loop, store *(frame timestamp, camera pose, intrinsics)* in a ring buffer covering about one second. When the result comes back, lift the ROI using the stored pose for that timestamp, producing a world-space position; the render loop then displays it with the *current* pose. Skipping this step is the most common cause of panels "swimming" behind the real object.

```
on_frame(frame):                      # render/AR thread, every frame
    ring.push(frame.ts, camera.pose, camera.intrinsics)
    if perception.idle() and quality_ok(frame): perception.submit(frame.cpu_image, frame.ts)

on_perception_result(res):            # worker thread -> posted to AR thread
    pose_t, K = ring.lookup(res.ts)             # pose when the pixels were captured
    Z = F * W_REAL[res.cls] / res.roi.width_px  # size prior (M3); or depth hit (M1/M2)
    p_cam = backproject(res.roi.center, Z, K)
    p_world = pose_t.transform(p_cam)
    object_tracker.update(p_world, res.confidence)   # smoothing + mode switch
```

6.5 Panel design and layout

- **Panels:** (1) identity card, (2) expiry bubble anchored near the printed date line (use that OCR box as its attach point), (3) warning chip, (4) action bar (Speak again / Save / Scan another). Limit to three visible panels to avoid clutter.
- **Billboarding:** panels face the camera; scale with distance so the *angular* size stays constant and text remains readable; enforce a minimum angular size.
- **Placement:** offset above or beside the object choosing the side with more free screen space; clamp inside the viewport; add hysteresis so panels do not flip sides every frame.
- **Leader lines** from a panel to its attach point on the object communicate "this belongs to that strip" without relying on depth cues.
- **Colour is never the only signal:** expiry uses number + word + icon + colour (for example "10 months · OK ✓", "12 days · Expires soon ⚠", "Expired ✕").
- **Occlusion (optional):** use depth or hand segmentation so panels do not draw over the user's fingers; skip in the MVP.

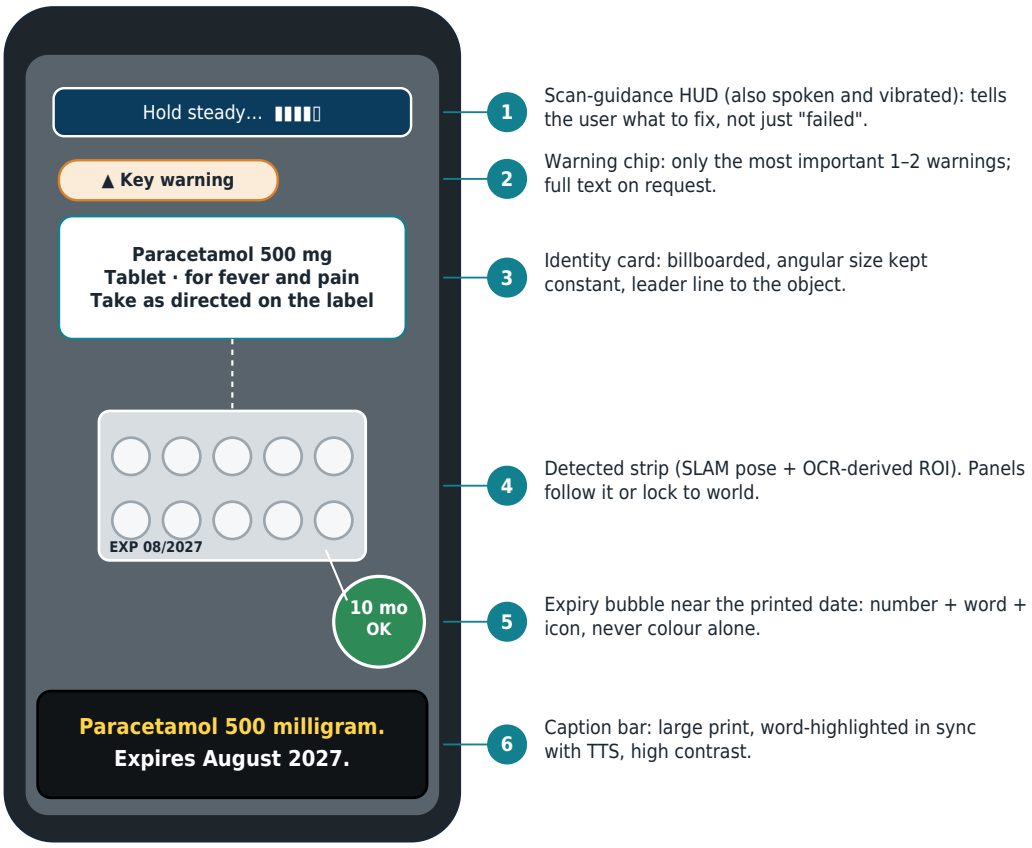


Figure 5. Overlay and caption layout (schematic). Numbered elements are explained at the right.

6.6 Tracking-failure handling

Signal from the AR session	User-facing response	System response
Tracking PAUSED: insufficient light	"It is too dark. Move to a brighter place."	Freeze overlays in ghost style; suspend OCR submit
Tracking PAUSED: excessive motion	"Moving too fast. Hold the phone steady."	Same; resume when tracking returns
Tracking PAUSED: insufficient features	"Show a bit more of the table or room behind the strip."	Switch to 2D HUD mode if it persists > 3 s
Tracking STOPPED / session error	"Something went wrong. Restarting."	Re-create session; keep the last result on screen as text

6.7 Performance and thermal management

- Run text detection only on stable, quality-passing frames; run any object detector at ~3–5 Hz; barcode scanning at a lower rate.
- Use hardware acceleration for models (GPU delegate / NNAPI-style runtimes) and keep model input sizes modest.
- Monitor thermal status; when it rises, lower perception frequency and AR frame rate before the OS throttles unpredictably.
- Budget battery explicitly in the evaluation (percentage per five minutes of scanning).

6.8 Curved bottle labels

A cylindrical label is only partly visible at once. Guide the user to **slowly rotate the bottle**; the tracker accumulates text lines from different viewpoints into the same object track (keyed by SLAM pose and text similarity). This is also a good "future work" item if the MVP handles only flat strips and cartons.

7. Accessibility Layer in Depth

Accessibility is not a mode that is added at the end; it shapes the interaction design. The principles: **eyes-free first**, **no precision gestures**, **redundant modalities** (speech + captions + haptics), **progressive disclosure**, **user control** (rate, voice, language, repeat), and **error tolerance**.

7.1 Scan-guidance engine (the missing piece in the overview)

Blind and low-vision users can hear and feel the app, so the app must tell them how to frame the object. The guidance planner turns measured conditions into short cues, rate-limited (for example no more than one spoken cue every ~2 s) and prioritised.

Condition detected	Speech cue	Haptic / sound cue
No text found	"I can't see any text. Hold the strip in front of the camera."	Slow pulse
Text found but too small	"Move a little closer."	Pulse rate up as distance improves
Text partly outside the frame	"Move the strip to the left." (direction from the box position)	Short tick
Glare detected	"There is glare. Tilt the strip slightly."	Double tick
Motion blur / shaky	"Hold steady."	None (avoid adding vibration while the phone is shaking)
Stable and readable	(silent) dwell timer starts	Rising tone
Recognition complete	Result speech begins	Strong confirm buzz + success earcon

7.2 Point-and-hold automatic trigger

The user never presses a button. Recognition starts when a **dwell condition** holds: gyroscope magnitude below a threshold, frame sharpness above a threshold, and text-token overlap (Jaccard) between consecutive frames above ~0.8, for about 1–1.5 s. A hardware-button trigger (for example volume key) remains as a backup. Dwell resets if the user moves.

7.3 Speech design

- **Order:** identity → expiry verdict → how the label says to take it → key warnings → common side effects. Summary first; "tell me more" for detail.
- **Wording:** short sentences, everyday words, numbers spoken in full ("five hundred milligram"), no medical jargon without a plain explanation.
- **Honesty:** "This reads as…" for auto-presented results; "I'm not sure" when confidence is low; always end with a brief reminder to follow the prescriber or pharmacist.
- **Control:** follow the system speech rate by default, allow faster/slower, allow the user to interrupt ("stop"), repeat ("repeat"), skip ("next").
- **Audio focus:** guidance cues and result speech never overlap; the result pre-empts guidance; duck other audio politely.

7.4 Synchronised large-print captions

Both Android TextToSpeech (range-start callbacks) and iOS AVSpeechSynthesizer (will-speak-range delegate) report which characters are about to be spoken, which allows **word-by-word highlighting**. Captions use large type (start at ≥ 24 sp equivalent), high contrast (≥ 7:1), no more than two lines at a time, and follow the user's font-scale setting.

7.5 Screen-reader compatibility (important and often missed)

WATCH OUT

AR content drawn by a GL/3D renderer is invisible to TalkBack and VoiceOver. Every overlay must therefore be mirrored into the normal accessibility tree: a live region that announces the result, labelled nodes for the identity, expiry and warnings, and standard buttons for Save / Repeat / Scan another. In the Compose-based approach (Section 10) the panels themselves are accessible UI elements positioned from projected anchors, which solves this almost for free.

7.6 Design guidance for elderly users

- Touch targets ≥ 48 dp (56 dp preferred); generous spacing; no tiny close buttons; no gestures that require precision.
- No timeouts that dismiss information; keep results on screen until the user leaves.
- Few steps: open app → hold strip → hear result. Settings are optional and reachable by voice.
- Voice onboarding: the first launch explains the app aloud and lets the user practise with a sample strip.
- Confirm before anything destructive (deleting history); undo where possible.

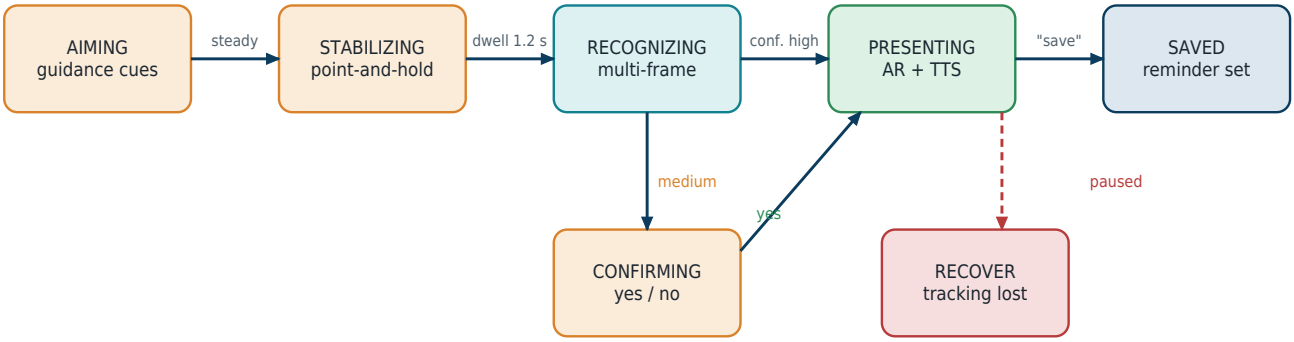
7.7 Voice commands

Keep the vocabulary tiny: *repeat, more detail, save, scan another, stop, slower, faster*. Use on-device recognition so the feature works offline and keeps audio private; a keyword-spotting approach is more robust than free dictation for such a small command set.

8. How the System Works, Functionally

8.1 End-to-end user journey

Step	What the user does / experiences	What the system does
1	Opens the app (or says the launch phrase). Hears "Ready. Hold a medicine strip in front of the camera."	Starts the AR session (autofocus on), loads the knowledge base into memory, enters **AIMING**.
2	Holds the strip about 20–30 cm from the phone.	Perception loop runs text detection on quality-passing frames; guidance engine monitors framing, distance, glare, tracking.
3	Hears cues such as "Tilt slightly — there is glare" and adjusts; feels ticks getting faster.	Cue planner maps measured conditions to speech and haptics; rate-limits.
4	Holds steady; hears a rising tone.	Dwell condition met → **STABILIZING** → **RECOGNIZING**; keeps fusing frames until fields are stable.
5	Feels a confirm buzz.	Fields extracted, matched, verified; confidence gate evaluates.
6a	High confidence: hears the result straight away.	**PRESENTING**: panels appear anchored to the strip; TTS reads the summary; captions highlight words.
6b	Medium confidence: hears "I think this is Metformin 500 milligram. Is that right?" and says "yes".	**CONFIRMING** state; accepts voice or large-button reply; "no" returns to AIMING with a tip.
7	Moves the strip: panels follow it. Puts it down: panels stay put.	Object tracker switches between object-following and world-locked modes.
8	Says "tell me more" or "repeat".	Plays the detail section or repeats the summary.
9	Says "save".	Stores product ID, confidence, expiry, timestamp; schedules reminders; confirms by voice.
10	Says "scan another".	Clears panels, returns to AIMING; recent history is kept.



Low confidence, "no" or timeout → back to AIMING with a spoken re-scan tip. "Scan another" from PRESENTING → AIMING.

Any state → RECOVER when ARCore tracking is PAUSED or STOPPED; returns to AIMING once tracking resumes.

Figure 6. Scan-session state machine.

8.2 Worked example (illustrative data)

The strings below are **made up to illustrate the data flow**, not measurements from a real package.

Stage	Content
Raw OCR, frame 1	"PARACETAM0L TABLETS IP 500 mg" · "EXP 08/2O27" · "B.No. PX2314" · "Mfd. by: ..."
Raw OCR, frame 4	"PARACETAMOL TABLETS IP 5OO mg" · "EXP. 08/2027"
After fusion (voting)	name: PARACETAMOL · form: TABLETS · strength: 500 mg · EXP: 08/2027 · batch: PX2314
Expiry logic	08/2027 → 31 Aug 2027; today = 15 Oct 2026 → about 10.5 months → status OK
Candidates	1) Paracetamol 500 mg tablet — score 0.97 (name ✓, strength ✓, composition agrees ✓) · 2) Paracetamol 650 mg tablet — 0.74 (strength ✗)

Stage	Content
Gate	Score ≥ 0.90, margin 0.23, strength verified → auto-present
Structured output	See JSON below
Spoken summary	"This reads as Paracetamol, five hundred milligram tablet. It is in date until August 2027. It is used for fever and mild to moderate pain. Follow your doctor's or the label's directions. Say 'tell me more' for warnings."

```
{
  "product_id": "P-000123", "match": {"score": 0.97, "margin": 0.23, "strength_verified": true},
  "read_from_label": {"name": "PARACETAMOL", "strength": "500 mg", "form": "tablet",
                      "expiry_raw": "EXP 08/2027", "batch": "PX2314"},
  "expiry": {"date": "2027-08-31", "days_left": 320, "status": "OK"},
  "content": {"summary_en": "...", "label_directions_en": "...", "warnings_en": ["..."],
              "side_effects_common_en": ["..."], "source": "<DB source>", "db_version": "<version>"},
  "gate": "AUTO_PRESENT"
}
```

8.3 Timing budget (illustrative targets, to be measured)

Phase	Target	Comment
Aim and stabilise	1–3 s	Depends on the user; guidance speeds this up
Fusion until stable	0.5–1.5 s	3–5 usable frames at a few Hz
Extraction + matching	< 50 ms	In-memory DB of a few thousand names is trivial
First speech	< 0.3 s after gate	Pre-warm the TTS engine at app start
Total from stable framing	≈ 2–3 s median	Proposed requirement NFR-1

8.4 Edge cases and alternative flows

Situation	Behaviour
Medicine is expired	Announce expiry first: "This appears to be expired in August 2025. Please do not use it; ask a pharmacist how to dispose of it."
Expiry not readable	Say so explicitly ("I could not read the expiry date. Please check the printed date.") — never assume in date.
Unknown medicine / not in DB	"I can read 'XYZ 250' but it is not in my database." Offer to save the raw reading, no clinical content.
Two medicines in view	Announce "I see two medicines — please show one at a time"; or lock to the one nearest the centre and most stable.
Repeated scan of the same item	Session cache: if the same product ID is stable again, skip re-announcing and offer "repeat".
TTS voice missing or engine unavailable	Fall back to large captions + haptics; prompt to install the offline voice.
Camera permission denied	Explain by voice and screen why it is needed; offer to open settings.
Tracking lost for a long time	Switch to 2D HUD mode (same recognition, no world anchoring).
Not a medicine	After repeated failure: "I don't think this is a medicine label."

8.5 Scan history and expiry reminders

- **Save** stores the product ID, confidence, parsed expiry, batch, timestamp and optionally a small cropped thumbnail (off by default for privacy).
- Expiry on labels is month-level, so treat it as the **last day of that month**.
- Schedule notifications at configurable offsets (default: 30 days, 7 days, on the expiry date) with a periodic background job as a safety net.
- Reminders must survive reboots: re-register scheduled work after device restart, and avoid relying on exact alarms (day-level accuracy is enough).

- Notification text uses the same plain-language style and is readable by screen readers.

9. Data and Knowledge Layer

9.1 Knowledge-base strategy: closed world first

A medicine identifier is only as trustworthy as its data. For a student project the realistic and defensible strategy is a **curated closed-world database** covering a few hundred of the most common medicines, with every record traceable to a source and reviewed. The system then says plainly when something is *not* in the database.

Tier	Scope	Content source	Effort
1 (MVP)	300–500 common medicines (start from an essential-medicines list for your target market, plus common brands)	Hand-curated from official labels/monographs; two-source cross-check; reviewer name and date	Medium
2 (optional)	Open-world lookup for names outside Tier 1	Online services (for example public drug-label APIs) with a clear "less verified" warning	Low–medium
3 (future)	Licensed commercial database	Commercial licence for brand-level data	Money, not code

9.2 Candidate data sources (check each licence)

Source	What it offers	Caveats
DailyMed / openFDA drug labels (US NLM / FDA)	Structured US prescribing information: indications, warnings, adverse reactions	US brands and wording; clinical-professional language, so it needs plain-language rewriting
MedlinePlus drug information / MedlinePlus Connect (US NLM)	Consumer-friendly drug summaries	US-oriented; verify reuse terms and keep attribution
RxNorm / RxNav (US NLM)	Normalised drug names, ingredients, strengths; approximate-name search	US product names; licensing details for some vocabularies
WHO essential-medicines and ATC classification	Generic names and therapeutic classes	Not brand-level; check reuse terms for ATC/DDD
India: national essential-medicines list and regulator approvals (public lists)	Backbone list of generic medicines to scope Tier 1; some brand/composition information	Not patient-friendly text; completeness varies
Community datasets of Indian brands and compositions (for example on Kaggle)	Brand → composition mapping for the Indian market	Quality and licence vary; no clinical text; validate before use
DrugBank	Rich relational drug data	Free for academic non-commercial use; commercial use requires a licence
Commercial Indian drug portals and APIs	Brand-level coverage	Typically not an open public API; scraping may violate terms — avoid

Statements about availability and licensing are from general knowledge and can change; confirm each source directly before use.

9.3 Two-hop model: brand → composition → monograph

Brand names differ between markets, but **active ingredient(s) + strength + form** is stable. Store products (brands) separately from ingredients and attach the patient-friendly monograph to the *ingredient* (or ingredient combination). A brand then inherits the monograph through its composition. This also makes brand-versus-generic matching a straightforward join rather than a fuzzy-string problem.

9.4 Matching in practice

- For a database of a few thousand names, **in-memory brute-force similarity** (Jaro-Winkler / Levenshtein over normalised names) runs in milliseconds on a phone; a full-text index is optional.
- Include aliases: brand spellings, common misspellings and OCR-typical corruptions, generated automatically with a confusion table (O↔0, l/I↔1, S↔5, B↔8, rn↔m, cl↔d, vv↔w).
- Tune weights and thresholds on **synthetic OCR noise** applied to DB names plus real scans; report the false-confident rate, not only accuracy.

- Keep a list of known look-alike/sound-alike pairs and force a confirmation when the top two candidates fall in one pair.

9.5 Content governance

- Every monograph field stores **source, retrieval date, reviewer and database version**; the app can show "Information version … ".
- Write at a plain-language reading level; avoid numbers that could be mistaken for a personal dose.
- Prefer "label says" phrasing and include a standing reminder to follow the prescriber or pharmacist.
- Have the content reviewed by a pharmacist or a qualified domain expert if at all possible; state in the report who reviewed what.

9.6 Expiry parsing rules

Rule	Detail
Formats	EXP 08/2027, EXP 08-27, EXP.12.2026, EXP AUG 2027, USE BEFORE Sep-2028, EXP 31/08/2027
Two-digit years	Interpret 20YY
Month-end	Expiry = last day of the printed month (unless a day is printed)
MFG vs EXP	Use the keyword. If two month-year values appear without keywords, take the later one as expiry but mark low confidence and confirm
Status bands (proposed)	Expired (< today) · Expires soon (≤ 30 days) · Caution (≤ 90 days) · OK (> 90 days)
Digit repair	Inside date fields map O→0, I/l→1, S→5, B→8 before parsing; reject month > 12
Out of scope	"Use within N months of opening" (liquids/eye drops) — flag as future work

10. Technology Stack — Brainstormed and Feasibility-Checked

The overview lists engines and libraries to *survey*. This section decides what to *build with*. The criteria: (1) shortest path to a working, accurate prototype, (2) first-class access to accessibility features (speech callbacks, haptics, screen readers), (3) strong on-device OCR, (4) enough AR capability for the design in Section 6, and (5) no licensing surprises.

10.1 Three candidate platform stacks

	A. Native Android (recommended)	B. Unity + AR Foundation	C. Native iOS
Stack	Kotlin, Jetpack Compose, ARCore, ML Kit, OpenCV, Room	Unity 6, AR Foundation, C#, native plugins for OCR/TTS or an on-device inference package	Swift, SwiftUI, ARKit, RealityKit, Vision, AVFoundation
Strengths	Direct camera-image-to-ML-Kit path; TalkBack/TTS/haptics are native; crisp accessible text; large device base for testing	One codebase for Android and iOS; rich 3D visuals; common in ARVR courses	Excellent text recognition; high-resolution frame capture; LiDAR on some devices; strong accessibility APIs
Weaknesses	Android only; 3D visuals need extra work (SceneView/Filament)	OCR, TTS and screen-reader integration need native bridges; text rendering and accessibility are harder; heavier build	Needs Apple hardware and a developer account for device testing; smaller device base in some markets
Best when	You want the most reliable working prototype	Your course mandates Unity or cross-platform	Your team already uses iPhones and Xcode

Weighted score (1–5; weights reflect this project's priorities and are my judgement — change them if your course requires Unity):

Criterion (weight)	A Android	B Unity	C iOS
Time to working prototype (25%)	5	3	4
OCR / ML integration (25%)	5	3	5
Accessibility API access (25%)	5	3	5
3D visual richness (10%)	3	5	4
Cross-platform reach (5%)	2	5	2
Fit with typical ARVR coursework (10%)	3	5	3
Weighted total	**4.45**	**3.50**	**4.30**

DESIGN DECISION

Recommendation: build on **Stack A** (or C if you work on iPhones). Choose B only if a Unity deliverable is required; in that case keep recognition in a native plugin and treat Unity purely as the AR/rendering shell. Keep the recognition core (parser, matcher, DB) in plain, platform-independent code so it can be ported.

10.2 Recommended stack by layer (Stack A, with iOS equivalents)

Layer	Primary choice	Why	Alternatives / iOS
Language / UI	Kotlin + Jetpack Compose	Accessible, declarative UI; easy to project 3D anchors into 2D panels	Swift + SwiftUI
SLAM / AR	ARCore (Android SDK)	Visual-inertial tracking, raycasts, planes, optional depth, tracking-failure reasons, session recording	ARKit (+ RealityKit); AR Foundation in Unity
AR rendering	Tier 1: Compose panels positioned by projecting anchor poses; Tier 2: SceneView (Filament) 3D nodes	Tier 1 gives crisp, screen-reader-friendly text; Tier 2 adds depth-aware 3D	RealityKit; Unity
Camera / images	ARCore camera image (YUV) with continuous autofocus; shared-camera path or largest config for high-res stills	Avoids fighting the AR session for the camera	ARKit frame + high-resolution capture
Image processing	OpenCV (Android SDK)	Laplacian sharpness, glare masks, homography/PnP, contour fitting	OpenCV iOS / Accelerate
OCR (primary)	ML Kit Text Recognition v2 (on-device, Latin)	Easy integration with camera images; good on clear print; free on-device	Apple Vision text recognition (accurate mode)
OCR (advanced)	PaddleOCR mobile models (detector + recogniser) via an on-device runtime, fine-tunable	Lets you fine-tune on your own foil-strip data; supports more scripts (Indic scripts if needed)	Tesseract (offline, weaker on scene text); docTR / EasyOCR for desktop prototyping
Cloud reading fallback (optional)	A vision-language model API, opt-in, crop only	Reads very hard labels; output is verified against the DB	Cloud OCR services
Barcode / DataMatrix	ML Kit Barcode Scanning	Supports 1D, QR and DataMatrix	ZXing; AVFoundation / Vision
Object detector (optional)	Small detector (YOLO-family or EfficientDet-Lite) exported to LiteRT; GPU delegate	Cleaner ROI and class prior for 3D lift	Core ML; MediaPipe Object Detector (Apache-2.0)
Hand tracking (optional)	MediaPipe Hands	Occlusion and finger-aware placement	Vision hand-pose
Matching	In-memory Jaro-Winkler / Levenshtein + phonetic codes over a few thousand names (SQLite full-text index optional)	Simplest thing that meets latency; fewer moving parts	Python RapidFuzz for offline tuning
Knowledge base	Pre-built SQLite shipped in assets; Room for access	Offline, versionable, easy to audit	Core Data / SwiftData; GRDB
Local history	Room + SQLCipher (encrypted)	Health-adjacent data should be encrypted at rest	SwiftData with file protection
Text-to-speech	Android TextToSpeech with range-start callbacks; offline voice packs	Word-level sync for captions; supports Indian English and several Indian languages when voices are installed	AVSpeechSynthesizer (will-speak-range delegate); offline neural TTS as an option (check licence)
Voice commands	On-device speech recognition, small command vocabulary (or Vosk keyword-style models)	Private, offline, robust for few commands	Apple Speech (on-device)
Haptics	VibrationEffect waveforms/primitives	Distinct patterns for guidance and success	CoreHaptics
Reminders	WorkManager + notifications; re-register after boot	Reliable day-level scheduling without exact-alarm permissions	UNUserNotificationCenter
Optional backend	FastAPI (Python) for dataset tooling and the opt-in fallback endpoint	Keeps the app offline-first	Serverless functions

10.3 ML, data and tooling

Purpose	Suggested tools
Model training / fine-tuning	Python, PyTorch, PaddleOCR training scripts, an object-detector training framework; Colab or a university GPU. Note: some popular detector frameworks use AGPL licences — fine for coursework, check before any commercial use
Annotation	Label Studio, CVAT or Roboflow for boxes and text; keep a CSV of ground-truth fields per scan
Augmentation / synthetic data	Albumentations for glare, blur, perspective and noise; script to render synthetic blister-strip text with specular highlights; OCR-confusion noise on DB names
Data and experiment tracking	DVC or plain versioned folders; Weights & Biases or a simple CSV log
Model deployment	Export to a mobile runtime (LiteRT / ONNX Runtime Mobile); quantise (INT8/FP16) and compare accuracy/latency
AR testing	ARCore session recording and playback: record camera + IMU sessions once, then replay deterministically to debug anchoring and measure drift
App testing	JUnit for parsers/matcher with golden OCR transcripts; Compose UI tests; Accessibility Scanner and manual TalkBack passes; Android Studio Profiler and system tracing for latency, battery and thermals
Design	Figma for panel and caption layouts; contrast checker for WCAG ratios

10.4 Device requirements and test matrix

- An ARCore-supported Android phone (API level high enough for TTS range callbacks, i.e. Android 8.0+); 4 GB RAM or more recommended; a phone with depth support is a bonus, not a requirement.
- Test on at least three tiers: low-end, mid-range, flagship — report FPS, OCR latency, battery drain and thermal behaviour for each.
- For iOS: iPhone with ARKit support; LiDAR models help depth but are optional.

10.5 Considered and not recommended (as the primary path)

Option	Reason
Pre-registered image targets for every medicine (ARCore Augmented Images, ARKit reference images, Vuforia targets)	Does not scale to open-ended packaging; glossy foil tracks poorly; Vuforia has licensing cost. Fine for a small demo set only
WebXR / browser AR	Limited SLAM support across browsers/devices; weak access to on-device OCR, TTS callbacks and haptics
Flutter / React Native AR wrappers	AR plugin maturity and accessibility control are weaker than native; extra bridging for ML
Tesseract as the only OCR	Weaker on scene text with glare; fine as a baseline in your comparison table
Generative model writing medical text	Unsafe: hallucination risk. Use verified DB text only (Section 4.2)
Head-worn AR glasses as the v1 device	Attractive for hands-free use, but hardware access and SDK maturity add risk; list as future work

11. Implementation Plan

11.1 Module structure (Stack A)

```
app/
  capture/      camera config, frame ring buffer, quality metrics (sharpness, glare, motion)
  ocr/          ML Kit wrapper, (optional) Paddle models, barcode reader
  fusion/       text-line tracking, token voting, stability logic
  extract/      regex + rules, expiry parser, strength/unit normaliser
  match/        normaliser, candidate retrieval, scorer, confidence gate
  kb/           SQLite asset, DAOs, content composer (templates, tall-man lettering)
  ar/           session manager, pose ring buffer, 2D->3D lift, object tracker, anchor manager
  ui/           Compose screens, projected panels, caption bar, accessibility mirror
  access/       guidance planner, dwell trigger, TTS queue + range sync, haptics, voice commands
  data/         history (encrypted), reminders (WorkManager), settings
tools/          python: dataset scripts, synthetic noise, evaluation harness, DB builder
```

11.2 Phased roadmap (assuming a ~12-week window — rescale to your calendar)

Phase	Weeks	Deliverables	Exit criterion
0 Scope and data start	1	Closed-world list (≈100 first), device choice, repo, consent-aware data collection plan, first 50 strip videos	Target medicines and metrics agreed
1 Recognition baseline	2–3	Offline harness: ML Kit OCR on recorded frames → extraction → DB v0; field-level accuracy numbers	Baseline report with failure categories
2 Fusion, matching, gate	4–5	Multi-frame fusion, brand+composition verification, confidence gate; DB to 300+	False-confident rate measured on validation set
3 AR integration	6–7	ARCore session, pose ring buffer, 2D→3D lift, world/follow modes, projected panels	Stable panels on a still strip and a hand-held strip
4 Accessibility	8–9	Guidance engine, dwell trigger, TTS + caption sync, haptics, TalkBack mirror, voice commands	A blindfolded teammate can scan and hear a result
5 Persistence and polish	10	History, reminders, 2D fallback, settings, onboarding	End-to-end flow without crashes on 3 devices
6 Evaluation and report	11–12	Ablations, user study, battery/thermal tests, literature gap analysis, final report and demo	Results table and demo video

11.3 MVP cut line

Must (the demo is incomplete without these)	Should	Could (stretch / future work)
ARCore tracking + OCR on strips/cartons; DB of ≥ 150–300 medicines; fusion; confidence gate; expiry parsing; AR panels in both modes; TTS + captions; guidance cues; history + reminders	Barcode channel; voice commands; 2D fallback; tall-man lettering; high-res still capture; user study	Custom detector; learned field tagger; hand occlusion; cylindrical bottle stitching; cloud fallback; multilingual speech; iOS port; spatial audio

WATCH OUT

Start data collection in week 1. OCR accuracy on foil is the main technical risk, and a representative dataset takes longer to gather than any code. Record 5–10 s videos of each medicine under varied lighting and angles (ARCore session recordings double as test fixtures). Split train/validation/test by *medicine and session* to avoid leakage.

12. Evaluation Plan

12.1 Metrics

Area	Metric	How to measure
OCR	Character error rate and word error rate on key fields; field-level exact match (name, strength, expiry)	Ground-truth CSV per scan; compare single-frame vs multi-frame
Extraction	Precision / recall / F1 per field; expiry parse accuracy	Annotated test set
Identification	Top-1 and top-3 accuracy; **false-confident rate**; confirmation rate; abstention rate	Closed-set test split; report per packaging type
AR stability	Jitter (px RMS) when still; drift (cm) after camera sweep; time-to-lock; tracking-loss rate	Tripod tests; recorded sessions; markers with known positions
Latency	Time to first result; per-stage timings; frame rate	On-device tracing on three device tiers
Resources	Battery % per 5 min; peak temperature; memory	System profiler; thermal status log
Accessibility	Task success rate; time on task; errors; SUS (System Usability Scale); NASA-TLX workload; subjective comfort	User study (below)

12.2 Experiments / ablations

ID	Question	Conditions
E1	Does multi-frame fusion help on foil?	Single-frame vs 3/5/10-frame voting
E2	Does quality gating + spoken guidance help?	Gate/guidance off vs on
E3	OCR engine comparison	ML Kit vs PaddleOCR (pretrained vs fine-tuned) vs Tesseract baseline
E4	Does composition cross-check reduce wrong-confident results?	Name-only vs name + composition + strength verification
E5	Does SLAM add value for placement?	2D tracking overlay vs SLAM world/follow modes; measure jitter, loss, user preference
E6	2D→3D lift methods	Raycast only vs size-prior vs combined, for hand-held objects
E7	Interaction design	Screen-result baseline app vs your eyes-free flow (task success and time)

12.3 User study design

- **Participants:** older adults and, where feasible and ethically approved, people with low vision or blindness (aim for at least 8–12 in total). Where recruiting is not possible, use clearly-labelled *simulated* impairment (blurring goggles, eyes closed) as a stop-gap and say so as a limitation.
- **Ethics:** obtain institutional approval or an exemption decision, informed consent, voluntary participation, no collection of personal prescription data; use purchased or donated sample packs, not patients' own prescriptions.
- **Tasks:** identify three unknown strips; find the expiry date of a strip; recover from a deliberately difficult (glare) strip; save a reminder.
- **Measures:** success, time, errors, number of guidance cues needed, SUS, NASA-TLX, qualitative interview.
- **Analysis:** small samples — use paired non-parametric tests where comparing conditions and report effect sizes and confidence intervals with caution.

13. Safety, Privacy, Ethics and Regulatory Notes

SAFETY
Not a substitute for a pharmacist or doctor. The app reads a label and shows general label-level information. It must never advise starting, stopping, skipping or changing a dose. Say this in onboarding, in the About screen, and briefly in spoken output.

- **Fail-safe behaviour:** abstain when unsure; confirm when close; show "reads as …" wording; expose the raw text that was read so the user (or caregiver) can compare.
- **Content integrity:** database-only medical text; versioned, sourced and reviewed; never generated on the fly.
- **Privacy:** on-device processing by default; history stored locally and encrypted; thumbnails off by default; clear delete-all control; explicit, per-use consent for any cloud fallback; request only the permissions needed (camera, notifications, microphone for voice commands). Medicine history is health-related personal data — consider applicable data-protection law in your market (for example India's Digital Personal Data Protection Act, 2023) and check current rules.
- **Regulatory awareness:** software that diagnoses or recommends treatment can be regulated as a medical device in several jurisdictions. A label-reading information aid that makes no diagnostic or treatment claims is a lower-risk category, but if you ever deploy beyond coursework, obtain proper regulatory and legal advice. Keep your claims modest in the report.
- **Inclusive testing:** test with real elderly and visually impaired users; do not assume that simulated impairment equals lived experience.
- **Bias and coverage:** state clearly which medicines, packaging types and languages are covered; avoid implying universal coverage.

14. Risk Register

Risk	Likelihood	Impact	Mitigation
OCR fails on foil glare / tiny print	High	High	Quality gate, multi-frame voting, guidance, high-res stills, fine-tuned recogniser, cloud fallback; start data collection early
Wrong confident identification (look-alike drugs)	Medium	Very high	Composition + strength verification, margin rule, LASA pair list, confirmation prompts; track false-confident rate
Panels swim or jitter on hand-held strip	High	Medium	Pose-at-capture compensation, smoothing, world/follow modes, ghost state when unsure
SLAM tracking loss near plain foil/palm	Medium	Medium	Keep background in view, map failure reasons to speech, 2D HUD fallback
Drug data coverage / licensing	Medium	High	Closed-world curated DB, sourced fields, clear "not in database" behaviour
Device heat and battery drain	Medium	Medium	Event-driven OCR, lower rates under thermal load, measure and report
Scope creep (detector, multilingual, iOS, interactions)	High	High	MVP cut line; staged upgrades; freeze scope at end of phase 3
No access to visually impaired participants	Medium	Medium	Partner with an organisation early; use simulated studies with explicit limitations
TTS voice / language availability varies by device	Medium	Medium	Detect missing voices, fall back to captions + haptics, guide installation
Misuse as medical advice	Low–Medium	High	Modest wording, standing disclaimers, no dose recommendations

15. Literature Survey Guide

This section turns your five survey threads into concrete reading tasks. **The works named below are well-known starting points recalled from general knowledge; verify every author, year and venue in Google Scholar before citing.** Search strings are given so you can find current work, including 2023–2026 papers I may not know.

Thread	Questions to answer	Starting points (verify)	Search strings
1 SLAM-based mobile AR	How accurate and robust is mobile VIO on small, shiny, hand-held targets? How do hybrid SLAM + marker/object-recognition systems work?	PTAM (Klein & Murray); ORB-SLAM family (Mur-Artal, Tardós et al.); VINS-Mono (Qin et al.); survey "Past, Present and Future of SLAM" (Cadena et al.); AR surveys (Azuma; Billinghurst et al.); ARCore/ARKit documentation	"visual-inertial odometry mobile AR accuracy", "markerless AR object anchoring", "hybrid marker SLAM AR"
2 OCR for pharmaceutical packaging	Accuracy on foil/glossy surfaces; glare handling; multi-frame text recognition; multilingual drug names	CRNN (Shi et al.); DBNet (Liao et al.); CRAFT (Baek et al.); PP-OCR (Du et al.); TrOCR (Li et al.); Tesseract (Smith); scene-text survey (Long, He, Yao)	"scene text recognition reflective surface", "pharmaceutical packaging OCR", "multi-frame text recognition video", "drug label text detection"
3 Drug information retrieval / matching	How are brand/generic names matched? How are OCR errors handled? Which data sources exist and what are their licences?	Approximate string matching surveys (Navarro); Levenshtein; Jaro-Winkler; phonetic algorithms; RxNorm / RxNav approximate matching; look-alike/sound-alike drug name literature (ISMP lists; tall-man lettering guidance)	"drug name matching OCR errors", "look-alike sound-alike drug names", "RxNorm approximate match"
4 AR + accessibility	What AR/audio systems exist for blind and low-vision users? How is aiming guidance done? What is known about older adults and AR?	VizWiz and blind-user photography work (Gurari et al.); VizLens (Guo et al.); SeeingVR (Zhao et al.); Google Project Guideline; Seeing AI and Be My Eyes (product descriptions and papers)	"augmented reality low vision", "blind users camera framing guidance", "older adults augmented reality usability"
5 Existing medicine identifiers	What do current apps do and not do? Which use barcodes, OCR, pill imprint/shape/colour, or reminders?	Pill-image recognition work (for example from NLM challenges); medication-reminder apps; reviews of medication-management apps; patient-safety reports (WHO Medication Without Harm)	"medicine identification mobile app OCR", "pill identification deep learning", "medication management app review accessibility"

15.1 Comparison table to build in your survey

Fill one row per system or paper. Suggested columns, with illustrative rows (fill with verified facts from the apps' current descriptions and papers):

System / paper	Input	Recognition	Output modality	AR / spatial anchoring?	Accessibility focus?	Medicine-specific?
Seeing AI (Microsoft)	Camera	Short text, barcode, scene	Audio	No (to verify)	Yes	Not medicine-specific
Be My Eyes / Be My AI	Photo / video	Volunteer or AI description	Audio	No	Yes	General
Google Lens	Photo	Text, objects	Screen result	No	Partial	General
Pill-identifier apps	Imprint / shape / colour entry or photo	Database lookup	Screen	No	Limited	Yes (loose pills)
Medication reminder apps	Manual entry / barcode	Lookup	Screen + notifications	No	Varies	Yes (scheduling)
This project	Live camera	OCR + barcode + fusion + verification	AR + speech + haptics	**Yes (SLAM-lifted)**	**Primary**	**Yes (packaged medicine)**

Entries for other systems are from general knowledge and must be confirmed against current app descriptions or papers.

15.2 Gap analysis and novelty statement (to refine after your search)

A defensible, appropriately modest statement: "*To the best of our knowledge, existing medicine-identification tools are screen-based scan-then-read applications, and general-purpose accessibility tools are not specialised for medicine packaging with verified, safety-gated output. We found no prior system that combines SLAM-lifted AR overlays with eyes-free, accessibility-first medication-label reading.*" Support it with a documented search protocol (databases, keywords, dates, inclusion criteria). Contributions you can claim and evaluate:

1. An **open-set, text-driven** method to attach AR content to arbitrary medicine packaging without pre-registered image targets.
2. **Multi-frame fusion + glare gating** for foil blister packs, with measured gains (E1, E2).
3. A **safety-gated identification** flow (cross-field verification, margin rule, confirmation) with the false-confident rate as a first-class metric (E4).
4. An **eyes-free scan-guidance and auto-trigger** interaction, evaluated with older or visually impaired users (E7).
5. A practical **hand-held 2D→3D lifting** recipe and a world/follow anchoring scheme (E5, E6).

15.3 Suggested limitations section

- Closed-world database coverage; labels in unsupported scripts; heavily worn or damaged print.
- Strongly reflective or curved packaging remains challenging; bottle labels need rotation.
- Small user study; simulated impairment if real participants are unavailable.
- Device dependence (ARCore/ARKit support, depth sensors, TTS voice availability).
- Informational purpose only; no personalised dosing; no interaction checking; no loose-pill identification.

16. Future Work

- **Loose pill identification** by imprint, shape and colour (dataset-heavy; separate pipeline).
- **Drug-interaction checking** across a user's saved medicines (needs clinical-logic validation).
- **Prescription reading** (printed and handwritten) with strong verification.
- **Multilingual output** and multilingual OCR for regional-language packaging.
- **Head-worn AR / smart glasses** for truly hands-free use.
- **Spatial audio** cues positioned at the object's anchor.
- **Caregiver sharing** with end-to-end encryption; refill and schedule integration with a user-entered prescription.
- **Authenticity checks** using supply-chain codes where available.
- **On-device small vision-language models** as a private fallback reader.

Appendix A — Data Schemas

A.1 Knowledge-base tables (SQLite, shipped as an asset)

```
CREATE TABLE ingredient (id INTEGER PRIMARY KEY, name TEXT NOT NULL, atc_code TEXT);
CREATE TABLE product (                       -- a brand / presentation
  id INTEGER PRIMARY KEY, brand_name TEXT NOT NULL, form TEXT, manufacturer TEXT, region TEXT,
  barcode_gtin TEXT, strength_text TEXT);
CREATE TABLE product_ingredient (
  product_id INTEGER REFERENCES product(id), ingredient_id INTEGER REFERENCES ingredient(id),
  strength_value REAL, strength_unit TEXT);
CREATE TABLE alias (text TEXT NOT NULL, product_id INTEGER, ingredient_id INTEGER, kind TEXT);
CREATE TABLE monograph (                     -- attached to ingredient (or fixed combination)
  ingredient_set_key TEXT, lang TEXT, purpose TEXT, label_directions TEXT, warnings TEXT,
  side_effects_common TEXT, side_effects_serious TEXT, storage TEXT,
  source TEXT NOT NULL, source_date TEXT, reviewer TEXT, db_version TEXT NOT NULL);
CREATE TABLE lasa_pair (a_ingredient_id INTEGER, b_ingredient_id INTEGER);  -- force confirmation
```

A.2 App-side tables (encrypted)

```
CREATE TABLE scan (
  id INTEGER PRIMARY KEY, ts INTEGER NOT NULL, product_id INTEGER, confidence REAL,
  gate TEXT,                 -- AUTO_PRESENT | CONFIRMED | UNRESOLVED
  expiry_date TEXT, batch TEXT, raw_text TEXT, thumbnail_path TEXT);   -- thumbnail optional
CREATE TABLE reminder (
  id INTEGER PRIMARY KEY, scan_id INTEGER REFERENCES scan(id), fire_at INTEGER, kind TEXT, status TEXT);
CREATE TABLE setting (key TEXT PRIMARY KEY, value TEXT);
```

Appendix B — Key Algorithms (tested snippets)

B.1 Expiry parser (Python; verified on the formats in Section 9.6)

Tested on: "EXP 08/2027", "Exp. AUG 2027", "EXP: 08-27", "EXP.12.2026", "Use before: Sep-2028", "EXP 31/08/2027", "MFG 09/2024 EXP 08/2027", "EXPIRY JAN. 28"; an invalid month (13) is rejected.

```
import re, datetime, calendar
MONTHS = {m.lower(): i for i, m in enumerate(calendar.month_abbr) if m}
EXP_RE = re.compile(r'''(?ix)
  \b(?:EXP(?:IRY)?\.?|EXPIRES?|USE\s*BEFORE|BEST\s*BEFORE)\s*[:\-]?\s*
  (?:  (?:(\d{1,2})\s*[/\-.]\s*)?(\d{1,2})\s*[/\-.]\s*(\d{4}|\d{2})        # 31/08/2027, 08/2027, 08-27
     | ([A-Za-z]{3,9})\.?\s*[-,/]?\s*(\d{4}|\d{2}) )                       # AUG 2027, Sep-2028
''')
def parse_exp(text):
    m = EXP_RE.search(text)
    if not m: return None
    _day, mo, yr, mon_txt, yr2 = m.groups()
    if mon_txt:
        mo, yr = MONTHS.get(mon_txt[:3].lower()), yr2
        if not mo: return None
    mo, yr = int(mo), int(yr)
    yr += 2000 if yr < 100 else 0
    if not 1 <= mo <= 12: return None
    return datetime.date(yr, mo, calendar.monthrange(yr, mo)[1])   # end of month

def status(exp, today):
    d = (exp - today).days
    return "EXPIRED" if d < 0 else "EXPIRES_SOON" if d <= 30 else "CAUTION" if d <= 90 else "OK"
```

Before parsing, apply digit repair *only inside date-like tokens* (O→0, I/l→1, S→5, B→8) so words are not corrupted.

B.2 Starting-point match score (tune on validation data)

```
score = 0.45*name_sim          # Jaro-Winkler / normalised Levenshtein on brand or generic name
      + 0.25*strength_match    # 1.0 exact, 0.0 mismatch (hard-fails if both sides present and differ)
      + 0.15*composition_agree # DB composition vs composition read from label (set overlap)
      + 0.10*form_match        # tablet / capsule / syrup ...
      + 0.05*maker_match       # weak confirming signal
if barcode_resolves_to(product) and composition_agree >= 0.8: score = 1.0
gate: AUTO if score >= 0.90 and (score - runner_up) >= 0.10 and strength_verified
      CONFIRM if score >= 0.70 (or margin small, or look-alike pair)    else RESCAN
```

Quick sanity check from testing: normalised Levenshtein similarity of "paracetam0l" vs "paracetamol" ≈ 0.91, while "metformin" vs "metronidazole" ≈ 0.31 — but short or similar names can still collide, which is why strength and composition are required.

B.3 Dwell (point-and-hold) trigger

```
stable_now = gyro_mag < G_MAX and sharpness > S_MIN and jaccard(tokens_t, tokens_prev) > 0.8
dwell = dwell + dt if stable_now else 0
if dwell >= 1.2 s: start_recognition()        # reset to 0 if the user moves; haptic tick at 0.6 s
```

B.4 Multi-frame voting (sketch)

```
for each text line L in frame:
    track = associate(L.box, tracks, camera_motion)         # IoU after motion compensation / flow
    track.samples.append((normalise(L.text), L.conf))
for track in tracks:
    cluster = strings within edit distance <= 1 of the modal string
    if len(cluster) >= 3 of last 5 usable frames:
        track.value = per_char_vote(align(cluster), weights=conf)   # fixes 0/O, 1/l, 5/S flicker
        track.stable = True
```

Appendix C — UI and Accessibility Specification (starting values)

Element	Specification
Caption text	≥ 24 sp (scales with system font size); bold; max two lines; current word highlighted; contrast ≥ 7:1
Caption background	Solid dark bar (≥ 85% opacity) so it stays legible over any camera content
Panel text	Minimum angular size constant across distances; sans-serif; 16 sp-equivalent minimum on screen
Expiry bubble	Number of months/days + word ("OK", "Soon", "Expired") + icon + colour; never colour only
Touch targets	≥ 48 dp (56 dp preferred), ≥ 8 dp spacing; primary action reachable with one thumb
Speech	Default to system rate; user adjustable; pre-warmed engine; interruptible; no overlap with guidance
Haptics	Distinct patterns: tick (framing), double tick (glare), rising pulses (closer), strong buzz (recognised)
Screen reader	Live region announces result; labelled nodes for each fact; Save / Repeat / Scan another as standard buttons
Motion / flashing	No flashing; fade transitions ≤ 250 ms; respect the system "reduce motion" setting
Onboarding	Spoken, skippable, repeatable; practice scan with a sample image

Appendix D — Glossary

Term	Meaning
SLAM / VIO	Simultaneous Localisation and Mapping / Visual-Inertial Odometry: estimating device pose (and a sparse map) from camera + IMU
6-DoF pose	Position (3) and orientation (3) of the camera or object
Anchor	A pose fixed in the SLAM world frame that the tracker keeps correcting over time
Raycast / hit test	Intersecting a ray from the camera with planes, feature points or depth to find a 3D position
Intrinsics	Camera focal length and principal point; needed to back-project pixels to 3D
PnP	Perspective-n-Point: solving an object's pose from known 3D points and their 2D projections
ROI	Region of interest in the image
LASA	Look-alike / sound-alike (drug names)
Tall-man lettering	Capitalising distinguishing parts of similar drug names (for example metFORMIN vs metroNIDAZOLE)
TTS	Text-to-speech
Earcon	A short, meaningful sound used as an interface cue
GS1 / DataMatrix	Standard product identification system and a 2D barcode format used on pharmaceutical packs
False-confident rate	Fraction of results that were announced as confident but were wrong — the key safety metric here